

Modeling trainings in Faculty of Medicine and Pharmacy of Casablanca as Constraint Satisfaction Problem

Amine Benamrane¹, Yosra Acodad¹, Imade Benelallam², El Houssine Bouyakhf¹, Mohammed Bennani Othmani³

¹ LIMIARF. Faculty of Sciences Mohammed the fifth University - Agdal Rabat, Morocco

benamraneamine@gmail.com, yosra.acodad@gmail.com, bouyakhf@fsr.ac.ma

² INSEA, National Institut of Statistics and Applied Economic - Irfane Rabat, Morocco

imade.benelallam@ieee.org

³ LIM. Faculty of Medicine and Pharmacy of Casablanca - Casablanca, Morocco
m.bennani@fmprc.ac.ma

Abstract. Hospital internships are essential and central component of any medical training to acquire different clinical skills, they require a commitment from the student and the supervisor.

In fact, the second cycle of medical studies in Faculty of Medicine and Pharmacy of Casablanca (*FMPC*) focuses on the acquisition of these clinical and therapeutic skills, and the encountered problem in the hospital courses management is adapting the number of students to home capacity, knowing that the assignment of students to hospital services must undergo certain constraints.

In effect, as the formalism of constraint satisfaction problem (*CSP*) can be simply, clearly and successfully adapted to a large class of real problems, we thought to use constraint programming (*CP*) to succeed this distribution of students per services.

In the present paper, we introduce the formalized *CSP*, then we present the component of the hospital training in the curriculum of the medical student, next we present our modeling of this distribution of students in the hospital services as a *CSP*. Finally, we conclude and propose some perspectives.

Keywords: CSP, modeling, medical Studies, allocation

1 Introduction

The constraint paradigm is a useful and well-studied framework expressing many problems of interest in Artificial Intelligence, operations research and other areas of Computer Science.

Constraint programming (*CP*) [1] distinguishes between the description of the constraints involved in a problem on the one hand, and the algorithms and

heuristics used to solve the problem on the other hand. Modeling and solving problems is through a very elegant mathematical formalism, called the constraint satisfaction problem (*CSP*) [2].

A significant progress has been made in *CP* in the last decade. In fact, many real problems, as scheduling, configuration, hardware verification, graph problems, molecular biology, etc..., can be formulated as constraint satisfaction problems (*CSPs*) in which solutions are assignments to a set of variables respecting a collection of constraints.

As formulating a good model (modeling the *CSP*) is of crucial importance to solving the model, we focus, in this paper, on modeling a real problem which is a hospital training.

The encountered problem is to allocate students of Faculty of Medicine and Pharmacy of Casablanca (*FMPC*) to some hospitals in the city, essentially to the university hospital complex (*UHC*) services while satisfying an imposed restrictions.

We note that, for solving this allocation problem, the *FMPC* administration already uses an application of students assignment management each year, but the obtained solution is always incomplete, then, the administration completes manually this distribution.

2 Background

2.1 Constraint satisfaction problem

The constraint programming is a very successful paradigm that can be used to model many real-world problems, as constraint satisfaction problems (*CSPs*), and solve them.

A model is an abstraction to a problem, that must respect the language of the solver. The quality of the model influences the quality of the solution.

A solution to a *CSP* is an assignments to a set of variables (each variable taking values from a certain domain), and in which there exists a collection of constraints that restrict the assignment of particular values or combination of values; solving a *CSP* means finding a feasible assignment of values to variables, i.e., one where all the constraints are satisfied.

Definition 1 (CSP). A Constraint Satisfaction Problem (*CSP*) is a tuple $\langle X, D, C \rangle$, where:

- X is a set containing n variables $\{x_1, x_2, \dots, x_n\}$.
- D is a set of domains $\{D(x_1), D(x_2), \dots, D(x_n)\}$ for these variables, with each $D(x_i)$ containing the possible values which x_i may take.
- C is a set of m constraints $\{c_1, c_2, \dots, c_m\}$ between variables in subsets of X . Each $c_i \in C$ expresses a relation defining which variable assignment combinations are allowed for the variables in the scope of the constraint, $\text{vars}(c_i)$.

A partial assignment is a set of tuple pairs, each tuple consisting of an instantiated variable and the value that is assigned to it in the current search node. A full assignment is one containing all n variables.

A partial solution is a consistent partial assignment, i.e., a partial assignment in which no constraints are violated. A solution to a *CSP* is a full assignment such that no constraint is violated.

2.2 Training in *FMPC*

Hospital internship is an essential part of the curriculum of the future medical doctor, each student in Faculty of Medicine and Pharmacy of Casablanca (*FMPC*) must pass this component to pass the academic year.

The administration of the *FMPC* must allocate students to the hospitals and to university hospital complex services respecting some constraints [3].

Actually, for solving this allocation problem, the *FMPC* administration uses, before the beginning of each year, an application of students assignment management, which is modeled using the formalism *OOP* (Object-oriented programming) [4] and developed with the *DotNet* platform [5]. For example, for a population of 500 students and about 20 services, with an average capacity of 55 beds per service, the current application takes between 7 and 10 days to declare a solution. However, the obtained result is always incomplete (some students remain without service because of the exhaustion of all services capacity), reason why the administration completes manually the distribution by violating some constraints.

Here are some interesting definitions which will be useful later:

Definition 2 (Period p). *A period p is a fixed period of one month and half (6 weeks).*

Definition 3 (complementary service). *A complementary service is a service that lasts one period.*

Cardiology, nephrology and dermatology services, are complementary service.

Definition 4 (Fundamental service). *A fundamental service is a service that occupies 2 periods. Otherwise, it is a couple $\langle FS_1, FS_2 \rangle$, where each component FS_i , such as $i \in \{1, 2\}$, lasts a single period.*

The pediatric service, for example, is a fundamental one. It is composed of *pediatric₁* and *pediatric₂*, and each component takes a single period.

Definition 5 (Obligatory service). *An obligatory service is a service whereby each student must pass.*

Definition 6 (Optional service). *An optional service is any service which is not obligatory.*

Either obligatory and complementary services are not necessarily fundamental services.

The problem constraints can be divided in two categories, the first one is the local constraints that manage the relationship of each student with its own six periods, and the second one is the global constraints that define the relationship between students and services in which they are assigned.

- Local constraints (concerning students):
 - each student must pass exactly six courses throughout the year (6 periods); (C1)
 - each student should not revisit a service; (C2)
 - each student must pass through a fixed number of obligatory services; (C3)
 - each student having a fundamental service s_i (part of the pair $\langle s_i, s_j \rangle$) in a given period t_k such as $k \in [1, 5]$, must be allocated in the following period t_{k+1} to the service s_j .(C4)
 - each student should not have a first part of a fundamental service in the latest course (sixth period).(C5)
- Global constraints (regarding services):
 - each service is characterized by its capacity that can not be exceeded.(C6)

3 modeling the problem of distributing students in the hospital services as a *CSP*

3.1 Motivation

We choose to formulate the problem of distributing students in the hospital services as a Constraint Satisfaction Problem (*CSP*) for the following reasons:

- The problem of assigning students per services is too complex to be solved manually;
- Many requirements on this assignation are naturally translated to constraints;
- Constraint Programming techniques have shown to be successful for applications similar to this one, such as resource allocation problems, scheduling problems,... etc.

3.2 Formulation of the *CSP*

The goal of this paper is to model the training problem already explained as a *CSP*.

To do this, we will translate the different components of the problem as a set $\langle Variables, Domains, Constraints \rangle$ such that the modeling describes exactly the problem.

The modeling can be done in several ways, we propose the one we see the clearest, the simplest and the most efficient.

Variables:

a *CSP* variable X_i^p is a period p for a given student i , such as: $i \in [1, number - of - students]$ and $p \in [1, 6]$. (C1)

we choose variables as above because solution must be a complete assignment of all periods for all students.

Domains:

Let $D = \{s_1, s_2, \dots, s_n\}$, the set of the n services in *UHC* (Universitary Hospital complex), be the domain of all *CSP* variables X_i^p .

$X_i^p = s_j$ means the student i is affected for his period p at the service s_j .

Constraints:

- Students' constraints: $\forall student i$,

C_i^1 :

$$\forall k, l \in [1, 6] \text{ such as: } k \neq l: \\ X_i^k \neq X_i^l \text{ (C2)}$$

C_i^2 :

S is the list of obligatory services and $m \in [1, cardinal(S)]$:

$$\sum_{p=1}^6 isIn(X_i^p, S) = m \text{ such as:}$$

$isIn(a, A)$ is the boolean function returning 1 when a is included in A . (C3)

C_i^3 :

$\forall S = \langle s, s' \rangle$ a fundamental service with the two unites s and s' ,

$$\forall p \in [1, 5]: \\ X_i^p = s \Rightarrow X_i^{p+1} = s' \text{ (C4)}$$

C_i^4 :

$\forall S = \langle s, s' \rangle$ a fundamental service with the two unites s and s' :

$$X_i^6 \neq s \text{ (C5)}$$

Ignoring constraint concerning students non obligatory courses does not negate their existence. In fact, due to their existence in variables domains and the high number of variables, a solution can not be found without using them.

Services' constraint:

C_s :

$$\forall s \in D, \forall p \in [1, 6]: occup_p(s) \leq capacity(s) \text{ (C6)} \\ \text{such as:}$$

$occup_p(s)$ is the number of students affected to the service s in p , one of the 6 periods.

$capacity(s)$ is the capacity of the service s that can't be exceeded.

4 Implementation

To measure the feasibility and rigor of our proposed modeling, we have implemented it on the *Choco* 2.0 platform [6] on a dual core machine with 1.72 *GHz* in processor and 2*GB* of *RAM*.

```

1. IntegerVariable[][] Students = Choco.makeIntVarArray(
   "Students",NbStudents,6,services,Options.V_ENUM); //(C1)
2. for (int i=0; i<NbStudents; i++) {
3.   CSPModel model = new CSPModel();
4.   model.addConstraint(Choco.allDifferent(Students[i])); //(C2)
5.   model.addConstraint(Choco.among(m,Students[i],ListOfObligatories)); //(C3)
6.   for (int j=0; j<NbOfPeriods-1; j++) {
7.     for (int l=0;l<ListOfFondamentals1.size();l++){
8.       model.addConstraint(Choco.implies(
   Choco.eq(Students[i][j],s1),Choco.eq(Students[i][j+1],s2))); //(C4)
9.       if (j==latest_period)
10.        model.addConstraint(Choco.notMember(
   Students[i][j],ListOfFondamentals1)); //(C5)
       }
     }
   }
11. model.addConstraint(Choco.occupationMax(k,Capacities.get(k),VarsOfStudents));
   //(C6)
12. CSPSolver solver = new CSPSolver();
13. solver.read(model);
14. solver.solve();

```

Fig. 1: Pseudo code

Students is the matrix of services of all students for all periods, otherwise, each *Student*[*i*][*j*] is a period of a student which takes a value from *services*, created by the function *makeIntVarArray* of the main class *Choco* (line 1). The procedure loops for all students (line 2) to define for each one its local constraints, and defines a model that contains all specifications of the current problem (line 3). Next, it fills the model by all local constraints (lines 4 to 10). After adding the global constraints concerning capacities of services (line 11), all constraints are added, then, the procedure defines the *CSPSolver* (line 12) and pass to it the model (line 13) before launching the solving process (line 14).

5 Conclusion and perspective

In this paper, we presented the feasibility of modeling the problem of allocating students of *FMPC* to various hospital services in the city as a *CSP*, according

to the requirements of the medical training program. As we showed through this work, the *CSP* framework is a very flexible and powerful model for this kind of problems.

As perspective, we intend to compare the efficiency and quality of this application as a *CSP* with its previous one, in terms of modeling and solving, and to develop this modeling using the *COP* (Constraint Optimization Problem) framework to optimize the occupation of all services by affecting the entire population.

References

- [1] R. Bartak, *Constraint Programming: In Pursuit of the Holy Grail*. In Proceedings of Week of Doctoral Students (WDS99), pp. 555-564, 1999.
- [2] E. Tsang, *Foundations of Constraint Satisfaction*. In Academic Press, London, 1995
- [3] T. Chkili, A. Harouchi, M. Berrade, A. Hasbi, *Decret N 2.91.527 DU 21 Kaada 1413 (13 mai 1993) Relatif la situation des externes, des internes et des residents des centres hospitaliers*, <http://www.enssup.gov.ma/index.php/textes-juridiques/statuts-particuliers/259-decret-n291527>. 13 may 1993.
- [4] B. Smith, *Object-Oriented Programming*. In AdvancED ActionScript 3.0: Design Patterns, Springer, pp. 1-25, 2011.
- [5] J. Reinfelds, *Programming as an engineering discipline*. In Frontiers in Education, 2002. FIE 2002. 32nd Annual, IEEE , pp. F2G-5, 2002.
- [6] C. Team, *Choco: an open source java constraint programming library*. In journal of Ecole des Mines de Nantes, Research report, pp. 10-02, 2010.